Vulnerability Assessment Report

AuSecurity External Vulnerability Assessment

AuSecurity
63 Paddington St
Paddington NSW 2021, Australia

Flinders University
Sturt Road, Bedford Park SA 5042

October 5, 2026

Report Version: 1.0

CONFIDENTIAL

Table of Contents

List of Figures 4

1 Document Control 5

1.1 Team 5

1.2 List of Changes 5

2 Executive Summary 6

2.1 Overview 6

2.2 Identified Vulnerabilities 7

3 Methodology 9

3.1 Objective 9

3.2 Scope 9

3.3 User Accounts and Permissions 10

4 Findings 11

C1: Unauthenticated osTicket Installer Exposed - Full Application Takeover 11

C2: SSH Password Authentication Enabled on All Three Hosts 13

H1: Backend Apache Server Exposed Directly on Port 8080 (osTicket) 15

H2: Email Domain Spoofing Trivial - Missing SPF, DKIM, DMARC and MX 17

H3: XML-RPC Interface Enabled With Pingback and system.multicall Methods 19

M1: Self-Signed TLS Certificate on Primary Website 20

M2: WordPress Login and XML-RPC Endpoints Reachable Without Rate Limiting 21

M3: Session Cookie Set Without HttpOnly, Secure, or SameSite Flags (osTicket) 22

M4: Directory Listing Enabled Across Multiple WordPress Paths 23

M5: Legacy IIS web.config File Served Publicly on Linux/nginx Host 24

M6: WordPress Admin Username Disclosure (User Enumeration) 25

M7: WordPress Install and Upgrade Endpoints Reachable (Already-Installed State) 27
M8: WordPress and Server Version Disclosure 28
M9: nginx 1.22.1 Running as Edge - Older Mainline Than Current Stable 30
M10: osTicket Setup Endpoint Also Discloses Internal Server Path 31
M11: No Security Response Headers on Primary Website 32
L1: Missing CAA DNS Record 34
5 Disclaimer 35
A Appendix 36
A.1 Static Appendix Section 36

List of Figures

Figure 1 - Distribution of identified vulnerabilities 8

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date
1.0	Initial report	Oct 5, 2026

2 Executive Summary

2.1 Overview

AuSecurity engaged an independent vulnerability assessment of its public internet-facing environment on 5 October 2026. The environment consists of three OVH (Singapore) virtual private servers hosting the marketing website (ausecurity.best), the WordPress blog (blog.ausecurity.best) and the osTicket customer-support portal (ticket.ausecurity.best). The attack surface is small, but it contains issues that materially affect the firm's security posture and brand.

Headline results

- **One critical issue** was identified: the customer-support portal (osTicket) is still showing its first-time installation page to the open internet. Anyone on the internet could complete that installer before AuSecurity does, and would then own the support platform. This should be fixed within hours, not days.
- **Three high-severity issues** were identified: (1) email sent as `@ausecurity.best` can be freely spoofed by any attacker because the domain publishes no email-authentication records, and the advertised contact address does not actually receive mail; (2) the backend web server behind the support portal is reachable from the open internet on an unencrypted port, bypassing the firm's own HTTPS protections; and (3) the main marketing website presents a self-signed TLS certificate, which triggers a full-page browser warning for every visitor and is especially damaging for a security-consulting brand.
- **Seven medium-severity issues** affect WordPress hardening (admin username is publicly discoverable, XML-RPC can be abused for amplification, several directories are publicly browsable), the support portal (information-leaking error messages, session cookies missing standard security flags), the marketing site (no security response headers), and all three servers (SSH still accepts password login).
- **Five low-severity issues** and **one informational observation** cover certificate-authority pinning, software version disclosure, a leftover Windows configuration file, missing rate-limiting on the WordPress login, an older nginx release, and WordPress install/upgrade endpoints that reveal state.

What this means for the business

There is no evidence that any of these issues has been exploited. However, the critical installer exposure allows total takeover of the support portal within minutes of discovery, and the email-authentication gap allows anyone to send convincing phishing messages that appear to come from AuSecurity staff. Both represent direct and immediate risk to the firm and its clients and should be remediated inside 72 hours. The remaining issues reflect common hardening gaps on newly deployed infrastructure; none require architectural change, and most can be resolved by configuration changes delivered in a single maintenance window.

Recommended priority order

1. Secure the osTicket installer (block `/setup/`, complete installation, close port 8080 to the public).
2. Publish SPF and DMARC records (and a working inbound mail path) for `ausecurity.best`.
3. Replace the self-signed certificate on the marketing site with a trusted (Let's Encrypt) certificate.

4. Harden WordPress (rename `admin`, restrict REST user enumeration, disable or rate-limit XML-RPC, remove directory listings).
5. Switch all three servers to SSH key-only authentication.
6. Add standard security response headers to all sites.
7. Address the remaining low and informational items as part of regular maintenance.

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
C1	10.0	Unauthenticated osTicket Installer Exposed - Full Application Takeover	11
C2	9.8	SSH Password Authentication Enabled on All Three Hosts	13
H1	8.2	Backend Apache Server Exposed Directly on Port 8080 (osTicket)	15
H2	8.2	Email Domain Spoofing Trivial - Missing SPF, DKIM, DMARC and MX	17
H3	7.3	XML-RPC Interface Enabled With Pingback and system.multicall Methods	19
M1	6.8	Self-Signed TLS Certificate on Primary Website	20
M2	6.5	WordPress Login and XML-RPC Endpoints Reachable Without Rate Limiting	21
M3	5.9	Session Cookie Set Without HttpOnly, Secure, or SameSite Flags (osTicket)	22
M4	5.3	Directory Listing Enabled Across Multiple WordPress Paths	23
M5	5.3	Legacy IIS web.config File Served Publicly on Linux/nginx Host	24
M6	5.3	WordPress Admin Username Disclosure (User Enumeration)	25
M7	5.3	WordPress Install and Upgrade Endpoints Reachable (Already-Installed State)	27
M8	5.3	WordPress and Server Version Disclosure	28
M9	5.3	nginx 1.22.1 Running as Edge - Older Mainline Than Current Stable	30
M10	5.3	osTicket Setup Endpoint Also Discloses Internal Server Path	31
M11	4.7	No Security Response Headers on Primary Website	32
L1	3.7	Missing CAA DNS Record	34

Vulnerability Overview

In the course of this penetration test **2 Critical**, **3 High**, **11 Medium** and **1 Low** vulnerabilities were identified:

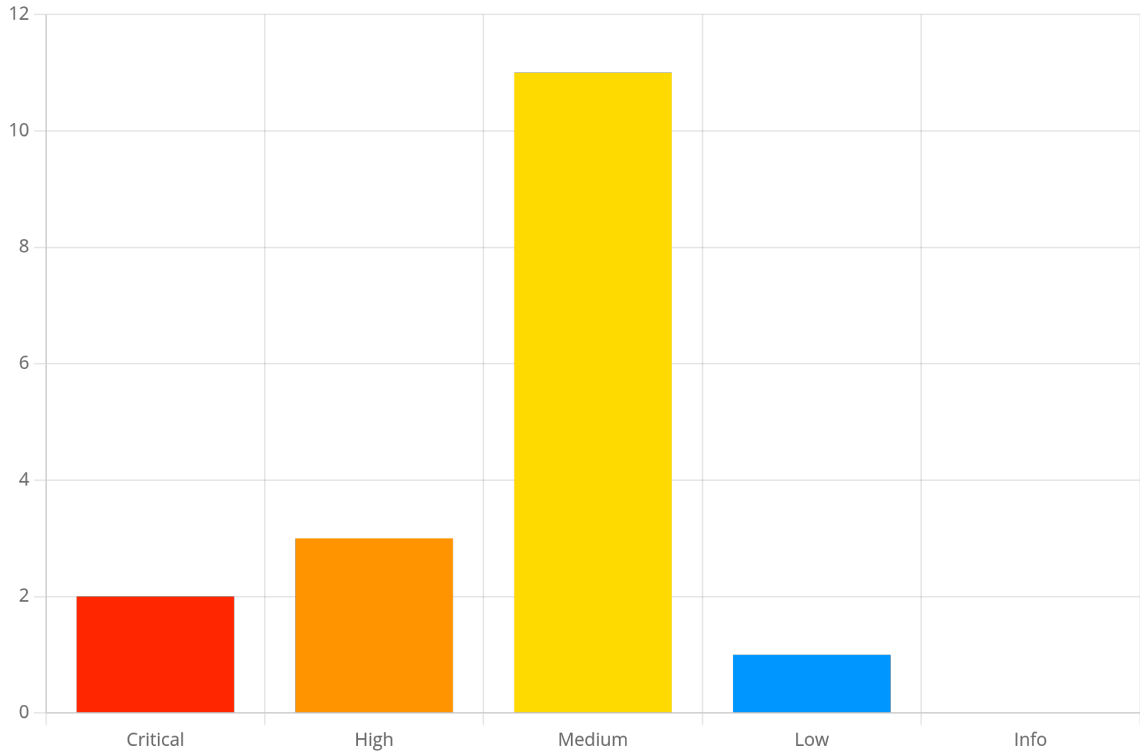


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasiable, seamless and cost-affective such approach is.

3.2 Scope

The engagement was a full-scope external vulnerability assessment of the publicly reachable AuSecurity environment, performed from the public internet with no prior credentials or internal network access.

In-scope assets

System	Description
ausecurity.best (apex)	Primary marketing website
www.ausecurity.best	Marketing website (www alias)
blog.ausecurity.best	WordPress blog
ticket.ausecurity.best	osTicket customer-support portal
51.79.166.148	OVH Singapore VPS hosting apex/www (ports tcp/22, 80, 443, 8080)
51.79.166.209	OVH Singapore VPS hosting ticket (ports tcp/22, 80, 443, 8080)
51.79.167.208	OVH Singapore VPS hosting blog (ports tcp/22, 80, 443)
ausecurity.best DNS / email posture	SPF, DKIM, DMARC, MX, MTA-STS, TLS-RPT, CAA, DNSSEC
info@ausecurity.best	Published public contact address

Methodology

Testing followed an adapted combination of PTES, OWASP Web Security Testing Guide (WSTG) and NIST SP 800-115. The following phases were executed:

- Passive and active enumeration (DNS, certificate transparency, subdomain discovery, Shodan hostname search).
- Infrastructure assessment (TCP port discovery, service and version fingerprinting, SSH configuration review).
- TLS inspection (certificate validation, chain verification, cipher review).

- DNS and email-posture review (SPF, DKIM, DMARC, MX, MTA-STS, TLS-RPT, CAA, DNSSEC, zone-transfer attempts).
- WordPress assessment (REST API, XML-RPC, author enumeration, plugin and theme fingerprinting, directory listing, admin endpoint reachability).
- osTicket assessment (installer exposure, source leakage, header hygiene, cookie flags, backend bypass).
- Cross-cutting web-application testing with templated checks covering common exposure, configuration, backup, disclosure, injection, SSRF, LFI, panel and login patterns.

Safety constraints applied throughout the engagement

- No credential bruteforce, password spraying or MFA bypass.
- No denial-of-service or destructive payloads.
- No data exfiltration or interaction with third parties.
- No social engineering or phishing.
- All exploitation probes stopped at minimum proof-of-concept.

Limitations

- Certificate-transparency enumeration via crt.sh was rate-limited; certspotter was used as a substitute. A follow-up crt.sh pass is recommended.
- No commercial API access was available for Censys, SecurityTrails, PassiveTotal or GitHub Code Search.
- The domain was first registered on 2026-08-04, so the Wayback Machine has no historical snapshots.
- The engagement was executed in a single day. Long-tail checks (large-wordlist content discovery, authenticated testing of osTicket once configured) were explicitly deferred.
- A full-template web scan timed out at approximately 49% and was replaced by a focused, tag-filtered re-run that completed successfully.
- No credential testing was performed, in line with the engagement rules.

Coverage summary

- 100% of enumerated in-scope hosts and web applications were probed.
- SSH, HTTP, HTTPS and HTTP-8080 were covered on every exposed host.
- The DNS and email posture of `ausecurity.best` was fully reviewed.
- WordPress core, REST API, XML-RPC, plugin and theme fingerprinting, directory listing and admin endpoints were all covered.
- osTicket installer exposure, source leakage, header hygiene, cookie flags and backend bypass were all covered.

3.3 User Accounts and Permissions

No user accounts, credentials or internal network access were provided. The assessment was performed entirely as an unauthenticated external attacker from the public internet.

4 Findings

C1: Unauthenticated osTicket Installer Exposed - Full Application Takeover	
Score	10.0 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Target	• https://ticket.ausecurity.best/setup/install.php • https://ticket.ausecurity.best/setup/ • http://51.79.166.209:8080/setup/install.php
References	• OWASP A05:2021 - Security Misconfiguration • osTicket Documentation: Post-Install Hardening (removal of /setup directory) • CWE-1188: Insecure Default Initialization of Resource

Overview

The osTicket 1.18.4 installer at https://ticket.ausecurity.best/ is publicly reachable and running in a fresh/uninitialized state. Every application path (/scp/, /tickets.php, /login.php, /api/, etc.) issues a 302 redirect to /setup/install.php, confirming the system has never been configured. An unauthenticated internet attacker can therefore walk through the installer, supply their own database connection and admin credentials, and gain full administrative control of the osTicket platform before the legitimate owner completes setup.

Details

Direct GET requests to https://ticket.ausecurity.best/setup/install.php return HTTP 200 with the "osTicket Installer" welcome page, including a PHP prerequisite screen (PHP 8.2.33, MySQLi loaded) and a "Continue" button that advances to database and administrator-account configuration. The same installer is also exposed in plaintext via the backend Apache (http://51.79.166.209:8080/setup/install.php) which responds identically regardless of the Host header, bypassing the nginx frontend entirely.

Business impact: an attacker completing the installer would obtain the first admin account and could then weaponise osTicket as a trusted inbound channel (phishing with legitimate ausecurity.best tickets, exfiltration of customer PII once real tickets are opened, deployment of a malicious knowledge-base article as a watering hole, and persistent code execution via plugins or email-piping scripts).

Supporting evidence collected during the engagement:

- P5-ticket/install-page-head.txt (installer welcome screen)

- setup/install.php)
- P5-ticket/8080-install-headers.txt (installer also reachable on plaintext 8080)
- P5-ticket/install-headers.txt

Recommendation

1. Immediately disable public reachability of /setup/ (nginx `location /setup/ { deny all; }` or filesystem removal `rm -rf /var/www/html/setup/` once installation is finished - per osTicket's official post-install hardening).
2. Complete the installation yourself with strong credentials before an attacker does.
3. Block external access to the Apache backend on 51.79.166.209:8080 at the host firewall / OVH network tab; only the nginx reverse proxy should reach it (loopback / private network).
4. Audit the osTicket database for any records created before you complete setup to detect prior attacker activity.

C2: SSH Password Authentication Enabled on All Three Hosts	
Score	9.8 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Target	• 51.79.166.148:22 • 51.79.166.209:22 • 51.79.167.208:22
References	• CIS Debian 12 Benchmark - SSH Hardening • CWE-521: Weak Password Requirements (potential), CWE-287

Overview

All three OVH VPS hosts expose OpenSSH 9.2p1 on tcp/22 and advertise `publickey,password` auth methods. Password authentication over SSH is the dominant source of hoster compromise in commodity VPS environments because of continuous large-scale credential-spraying from botnets. The hosts have no fail2ban-visible behaviour (RST-ratelimit was observed on nmap but not login-attempt limiting).

Details

`nmap --script ssh-auth-methods` reported `Supported authentication methods: publickey, password` on all three IPs. OpenSSH version is current for Debian 12 stable (2+deb12u10). No weak key exchanges or ciphers were offered, but HMAC-SHA1 variants (`hmac-sha1`, `hmac-sha1-etm@openssh.com`) are still available.

Business impact: a single reused or guessable password on root or on a sudoer account grants shell on the host - which contains the WordPress blog, the osTicket instance, and the website source. Password auth also removes any benefit from the OVH-native 2FA on the control panel.

Supporting evidence collected during the engagement:

- P1-ssh/148-audit.txt
- P1-ssh/209-audit.txt
- P1-ssh/208-audit.txt

Recommendation

1. Switch to key-only auth: in `/etc/ssh/sshd_config` set `PasswordAuthentication no`, `PermitRootLogin prohibit-password`, `KbdInteractiveAuthentication no`, then `systemctl reload sshd`. Verify your keys work from a second terminal before closing the first session.
2. Disable the weak MACs: `MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com,umac-128-etm@openssh.com`.
3. Install `fail2ban` with the sshd jail enabled, or move SSH behind a WireGuard tunnel / Tailscale.

4. Consider restricting :22 to a management IP list at the OVH firewall.

H1: Backend Apache Server Exposed Directly on Port 8080 (osTicket)	
Score	8.2 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
Target	http://51.79.166.209:8080/ (ticket.ausecurity.best backend)
References	• OWASP A05:2021 - Security Misconfiguration • CWE-668: Exposure of Resource to Wrong Sphere

Overview

The osTicket backend Apache 2.4.68 instance is directly reachable from the public internet on tcp/8080 without TLS. This bypasses the nginx reverse proxy on 443 and permits identical access to the osTicket installer and application in plaintext. Any security controls that rely on the nginx edge (TLS, future WAF rules, HSTS, request-size limits, rate-limits) can be evaded by connecting to port 8080 directly.

Details

nmap confirmed 51.79.166.209:8080 open, running Apache/2.4.68 (Debian). The backend responds identically to IP:8080 and to Host: ticket.ausecurity.best, with no virtual-host separation. GET http://51.79.166.209:8080/ returns 302 to /setup/install.php (the installer takeover vector described earlier), and the installer page is fully usable over plaintext HTTP. Positive controls observed on the backend: /server-status returns 403, /server-info returns 404, /.htaccess returns 403, and /include/ost-config.php returns 403.

Business impact: beyond compounding the installer takeover risk by removing TLS, this exposure means credentials entered into the live osTicket admin login (once configured) can be captured by any passive observer on the network path if a client happens to be directed - or redirected - to port 8080.

Supporting evidence collected during the engagement:

- P1-portscan/209-top1000.nmap
- P1-sv/sv-all.nmap
- P5-ticket/8080-root.txt
- P5-ticket/8080-install-headers.txt
- P5-ticket/8080-host-ticket.txt

Recommendation

1. Block tcp/8080 at the OVH network firewall or on the host `iptables -I INPUT -p tcp --dport 8080 ! -s 127.0.0.1 -j DROP`.

2. Reconfigure Apache to bind to 127.0.0.1:8080 only (`Listen 127.0.0.1:8080` in /etc/apache2/ports.conf) so it is impossible to expose externally.
3. Verify similarly on the apex host (51.79.166.148:8080 also exposes a backend nginx 1.25.5 serving identical content).

H2: Email Domain Spoofing Trivial - Missing SPF, DKIM, DMARC and MX	
Score	8.2 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:H/A:N
Target	ausecurity.best (and all subdomains - no subdomain policy published)
References	• RFC 7208 (SPF), RFC 6376 (DKIM), RFC 7489 (DMARC) • M3AAWG Email Authentication Recommendations • CWE-290: Authentication Bypass by Spoofing

Overview

The ausecurity.best DNS zone publishes no SPF, DKIM, DMARC, MX, MTA-STS, or TLS-RPT records. Any attacker on the internet can send email that appears to come from any @ausecurity.best address (including info@, owner@, admin@) and recipient mail servers have no cryptographic or policy signal to reject it. In parallel, the published contact address info@ausecurity.best is undeliverable because there is no MX record pointing at a mail server.

Details

Direct DNS queries confirm complete absence of email-authentication records:

- MX ausecurity.best: no answer
- TXT ausecurity.best (SPF): no v=spf1 record
- TXT _dmarc.ausecurity.best: no answer
- TXT ._domainkey.ausecurity.best (tested: default, google, selector1, selector2, mail, s1, s2, k1, dkim): no answer
- TXT _mta-sts.ausecurity.best / _smtp._tls.ausecurity.best: no answer

Business impact: an attacker can impersonate AuSecurity in Business Email Compromise (BEC) attacks against clients, suppliers, Flinders University contacts, or prospective customers. Because AuSecurity publicly positions itself as a GRC consulting and penetration-testing firm, successful spoofing is particularly damaging to brand trust and could also be used to lure the firm's own clients into disclosing sensitive data. The absence of an MX record also means customers who email info@ausecurity.best (as advertised on the website contact page) receive no reply, which is a direct revenue/reputation issue.

Supporting evidence collected during the engagement:

- P3-dns/mx.txt
- P3-dns/spf.txt
- P3-dns/dmarc.txt
- P3-dns/dkim.txt

- P3-dns/mtasts.txt

Recommendation

1. Publish a strict SPF record. If no mail is sent from the domain today: `v=spf1 -all`.
2. Publish a strict DMARC policy: `v=DMARC1; p=reject; sp=reject; adkim=s; aspf=s; rua=mailto:dmarc-reports@ausecurity.best; fo=1`.
3. If/when real mail is enabled: set up a provider (Google Workspace / M365 / Fastmail), publish MX records, enable DKIM signing with per-selector 2048-bit keys, and tighten SPF to the provider's include list.
4. Consider MTA-STS (`v=STSv1; id=...; mode=enforce; mx=...; max_age=86400`) and TLS-RPT once mail flow is established.
5. Until mail is set up, replace the published info@ausecurity.best contact with a working channel (e.g., a web form on the marketing site posting into osTicket after it is secured).

H3: XML-RPC Interface Enabled With Pingback and system.multicall Methods	
Score	7.3 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L
Target	https://blog.ausecurity.best/xmlrpc.php
References	• WordPress XML-RPC Security Guidance • CVE-2017-6815 class (xmlrpc amplification), CWE-346

Overview

WordPress XML-RPC is enabled and responds to `system.listMethods`, `pingback.ping`, `wp.getUsersBlogs`, `wp.getUsers`, and the batching method `system.multicall`. These collectively enable (a) brute-force credential attacks that are rate-limit-evasive via multicall, (b) pingback-based DDoS amplification against third parties, and (c) further user enumeration.

Details

POST of `<methodCall><methodName>system.listMethods</methodName>...</methodCall>` returned ~80 methods including the three attack-amplifying ones listed above. A pingback.ping test produced a parsed XML-RPC fault response, confirming the method is live.

Business impact: WordPress sites with XML-RPC enabled are routinely abused as participants in DDoS reflection; being used as a reflector exposes the business to abuse complaints, hoster action and potential blocklist listings that affect the main domain's reputation.

Supporting evidence collected during the engagement:

- P4-blog/xmlrpc-methods.txt
- P4-blog/xmlrpc-headers.txt
- P4-blog/xmlrpc-pingback.txt

Recommendation

1. If XML-RPC is not needed (Jetpack / mobile Classic App not in use), block it at the web server: `location = /xmlrpc.php { deny all; return 403; }` (nginx) or Apache `<Files xmlrpc.php> Require all denied </Files>`.
2. If it must stay enabled, disable at least pingback via the `xmlrpc_methods` filter: `add_filter('xmlrpc_methods', fn($m)=>array_diff_key($m, ['pingback.ping'=>0,'pingback.extensions.getPingbacks'=>0]));`
3. Rate-limit /xmlrpc.php at the edge (nginx `limit_req_zone`).

M1: Self-Signed TLS Certificate on Primary Website	
Score	6.8 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:H/A:N
Target	• https://ausecurity.best/ • https://www.ausecurity.best/
References	• RFC 5280 (X.509), CA/B Forum Baseline Requirements • CWE-295: Improper Certificate Validation (consequence)

Overview

The TLS certificate on https://ausecurity.best/ (CN=ausecurity.best, issuer=ausecurity.best) is self-signed. Every modern browser displays a full-page security warning, and users who proceed past the warning are effectively training themselves to accept certificate errors. Attackers performing TLS interception against site visitors face no certificate-pinning or CA-trust barrier because the baseline cert is already untrusted.

Details

`openssl s_client -connect ausecurity.best:443 -servername ausecurity.best` returns a certificate whose issuer and subject are both `CN=ausecurity.best`, validity 2026-09-14 to 2027-09-14, no CA chain. The nginx front proxy returns the same self-signed cert whether SNI is used or not, and the same certificate is served when connecting to the IP 51.79.166.148:443 directly. Blog and ticket subdomains are correctly covered by Let's Encrypt-issued certificates - only the primary marketing domain is affected.

Business impact: as a GRC and penetration-testing firm, a browser warning on the home page undermines the firm's credibility and reduces visitor conversion. More concretely, it conditions prospective clients to click through certificate errors, which is a direct enabler of phishing against the firm's clientele.

Supporting evidence collected during the engagement:

- P2-apex/cert-apex.txt
- P2-apex/cert-nosni.txt
- P2-apex/cert-ip.txt
- P1-sv/sv-all.nmap

Recommendation

1. Issue a free Let's Encrypt certificate for ausecurity.best + www.ausecurity.best (`certbot --nginx -d ausecurity.best -d www.ausecurity.best`). Automate renewal via systemd timer or cron.
2. Publish a CAA record (see separate finding) once the issuing CA is chosen.
3. Enable HSTS (see Missing Security Headers finding) once a trusted cert is in place.

M2: WordPress Login and XML-RPC Endpoints Reachable Without Rate Limiting	
Score	6.5 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
Target	• https://blog.ausecurity.best/wp-login.php • https://blog.ausecurity.best/xmlrpc.php
References	• OWASP WSTG-ATHN-04 • CWE-307: Improper Restriction of Excessive Authentication Attempts

Overview

Both authentication entry points on the blog respond with HTTP 200 to unauthenticated requests without any evidence of rate-limiting, lockout, CAPTCHA, or source-IP throttling. Combined with the disclosed `admin` username and lack of 2FA, credential attacks against the blog are possible. (We did NOT run a credential attack - the engagement rules forbid credential bruteforce.)

Details

GET /wp-login.php returns 200 and the standard login form with `wordpress_test_cookie` set. GET /xmlrpc.php returns 200 and the "XML-RPC server accepts POST requests only" message; POST yields full RPC. There is no observable rate-limit header, no fail2ban-style 429 response under repeated hits, and no 2FA indication in the login HTML.

Business impact: credential-spraying from botnets is continuous for every public WordPress login; the probability of eventual success on an `admin` account grows over time without MFA / lockout.

Supporting evidence collected during the engagement:

- P4-blog/wplogin-headers.txt
- P4-blog/xmlrpc-headers.txt

Recommendation

1. Enforce multi-factor authentication (plugins: Wordfence, miniOrange 2FA, Two Factor).
2. Rename or restrict /wp-login.php (plugin: WPS Hide Login) OR add nginx `allow`/`deny` for the admin management IP range.
3. Enable login lockout + CAPTCHA (Wordfence, Limit Login Attempts Reloaded).
4. Rate-limit the endpoints at nginx: `limit_req_zone $binary_remote_addr zone=login:10m rate=10r/m;` then `location = /wp-login.php { limit_req zone=login burst=3 nodelay; }` and similarly for /xmlrpc.php.

M3: Session Cookie Set Without HttpOnly, Secure, or SameSite Flags (osTicket)	
Score	5.9 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:L/A:N
Target	https://ticket.ausecurity.best/* (PHPSESSID cookie)
References	• OWASP WSTG-SESS-02, OWASP Secure Cookie Attributes • CWE-614, CWE-1004, CWE-1275

Overview

PHPSESSID cookies issued by osTicket (both via nginx:443 and direct Apache:8080) are set with only `path=/`. The HttpOnly, Secure and SameSite attributes are all missing. If a cross-site-scripting flaw is later found (or already exists) anywhere in osTicket, the session cookie can be stolen via JavaScript. The cookie can also be sent in plaintext over port 8080 and leaked in cross-site contexts.

Details

Observed Set-Cookie header on /setup/install.php response: `Set-Cookie: PHPSESSID=ecacfde8cdb7c6e1a188972075364a98; path=/` No HttpOnly, no Secure, no SameSite. nmap -sC also raised the http-cookie-flag ("PHPSESSID flag not set: httponly, secure").

Business impact: once osTicket is in production, a single XSS - a very common class in support-ticket applications where users submit free-form HTML / attachments - would convert into immediate agent-session hijack.

Supporting evidence collected during the engagement:

- P5-ticket/install-headers.txt
- P1-sv/sv-all.nmap

Recommendation

Configure the PHP session or the web server to add secure cookie attributes:

- PHP (php.ini or FPM pool): `session.cookie_httponly = 1` `session.cookie_secure = 1` `session.cookie_samesite = "Lax"`
- Or set via nginx: `proxy_cookie_flags PHPSESSID httponly secure samesite=lax;`

M4: Directory Listing Enabled Across Multiple WordPress Paths	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	• https://blog.ausecurity.best/wp-content/uploads/ • /wp-content/uploads/2026/ • /wp-includes/
References	• CWE-548: Exposure of Information Through Directory Listing • Apache mod_autoindex guidance

Overview

Apache's `mod_autoindex` is enabled on three paths on blog.ausecurity.best, allowing anyone to enumerate uploaded media filenames and the internal WordPress source tree. Attackers use directory listings to find backup files, orphaned media, draft content and to confirm plugin/theme file layouts for targeted CVE exploitation.

Details

- GET /wp-content/uploads/ returns the standard Apache "Index of" page showing `2026/`.
- GET /wp-content/uploads/2026/ shows `09/` and `10/` subdirectories.
- GET /wp-includes/ fully lists core WordPress source files and directories (`ID3/`, `IXR/`, `PHPMailer/`, `Requests/`, `SimplePie/`, `abilities-api.php`, `admin-bar.php`, etc.).

Business impact: enables file-targeted probing and often reveals forgotten assets (customer PDFs, draft images, exported DBs placed in /uploads/ by backup plugins).

Supporting evidence collected during the engagement:

- P4-blog/common-paths.txt
- P6-verify/uploads-root.html
- P6-verify/uploads-2026.html
- P6-verify/wp-includes.html

Recommendation

1. Disable autoindex globally: in `/etc/apache2/apache2.conf` ensure `<Directory /var/www/>` contains `Options -Indexes`.
2. Alternative per-vhost: add `Options -Indexes` or create empty `index.html` files in each listed directory as a temporary mitigation.
3. Audit `/wp-content/uploads/2026/09/` and `/10/` for any accidentally uploaded sensitive files.

M5: Legacy IIS web.config File Served Publicly on Linux/ nginx Host	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://ticket.ausecurity.best/web.config
References	CWE-538: File and Directory Information Exposure

Overview

A Microsoft IIS/ASP.NET `web.config` file is served from the root of ticket.ausecurity.best. The server is nginx + Apache + PHP on Debian, so the file is inert, but publishing it discloses that the system or its image was prepared from a Windows template and may indicate reused configuration artefacts elsewhere.

Details

GET /web.config returns 200 with a valid IIS URL Rewrite XML document (no credentials, no connection strings). The file is not consulted by nginx or Apache. Its presence suggests either a) the deployment was copied from a Windows origin, or b) the osTicket image carries leftover vendor files - either way worth cleaning up.

Supporting evidence collected during the engagement:

- P6-verify/web-config.txt

Recommendation

1. Delete the file from the web root.
2. Audit the webroot for other Windows/IIS artefacts (`*.aspx`, `App_Data/`, `bin/`) and the Apache webroot for similar leftover vendor files.

M6: WordPress Admin Username Disclosure (User Enumeration)	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://blog.ausecurity.best/
References	• OWASP WSTG-IDNT-04 • CWE-200: Exposure of Sensitive Information to an Unauthorized Actor

Overview

The WordPress REST API, the legacy `?author=N` redirect, and the `wp-links-opml.php` endpoint all expose the administrator username `admin` without authentication. This provides half of the credentials needed for a brute-force or credential-stuffing attack against /wp-login.php and the XML-RPC endpoint.

Details

- GET /?rest_route=/wp/v2/users returns a JSON array disclosing user_id=1, name="admin", slug="admin", avatar hash.
- GET /?author=1 responds with 301 to /author/admin/ (classic WordPress user-enum).
- GET /wp-links-opml.php returns XML containing `<!-- generator="WordPress/7.1.2" -->` and author metadata.

Business impact: narrows attack surface for a credential attack and signals a low-maturity WordPress hardening posture.

Supporting evidence collected during the engagement:

- P4-blog/wpjson-users.txt
- P4-blog/author-enum.txt
- P4-blog/rest-users.txt
- P6-verify/wp-links-opml.txt

Recommendation

1. Rename the primary administrator account from `admin` to a non-guessable value (create a new admin, log in as it, delete `admin` and reassign posts).
2. Restrict /wp-json/wp/v2/users to authenticated requests, e.g. via the WPCode snippet: `add_filter('rest_endpoints', fn($e)=>(unset($e['/wp/v2/users'],$e['/wp/v2/users/(?P<id>[\d]+)'])) ?: $e));`
3. Block `?author=N` enumeration (plugin "Disable Author Pages" or an nginx `rewrite`).

4. Deny direct access to `wp-links-opml.php` and `readme.html` via server config.

M7: WordPress Install and Upgrade Endpoints Reachable (Already-Installed State)	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	• https://blog.ausecurity.best/wp-admin/install.php • /wp-admin/upgrade.php
References	CWE-200

Overview

Unauthenticated requests to /wp-admin/install.php and /wp-admin/upgrade.php return HTTP 200 showing "Already Installed" and "No Update Required" respectively. Unlike the osTicket case these endpoints do not allow re-installation, but their reachability leaks state and provides low-value reconnaissance.

Details

Body of /wp-admin/install.php contains "Already Installed - You appear to have already installed WordPress. To reinstall please clear your old database tables first." /wp-admin/upgrade.php shows "No Update Required". These confirm the WP installation is complete and current with its DB schema.

Supporting evidence collected during the engagement:

- P4-blog/install-body.html
- P4-blog/upgrade-body.html

Recommendation

Deny unauthenticated access via nginx:

```
location ~* /wp-admin/(install|upgrade)\.php$ { allow <admin IP>; deny all; }
```

or delete `install.php` after install (WordPress supports this).

M8: WordPress and Server Version Disclosure	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	• https://blog.ausecurity.best/ (WordPress 7.1.2 • All-in-One WP Migration 7.109 • Akismet 5.7.2 • Hello Elementor 3.0.0) • https://ausecurity.best/ (nginx 1.22.1 and 1.25.5) • https://ticket.ausecurity.best/ (nginx 1.22.1 • Apache 2.4.68 • PHP 8.2.33 • osTicket 1.18.4)
References	• CWE-200, CWE-205 • OWASP WSTG-INFO-02

Overview

Every host leaks precise software and version data via one or more of: Server header, X-Powered-By header, generator meta tag, /readme.html, /license.txt, /wp-links-opml.php, and plugin readme.txt files. This lets attackers map which published vulnerabilities apply without any probing.

Details

Confirmed version strings:

- blog: WordPress 7.1.2 (/readme.html, generator meta, /wp-links-opml.php), Hello Elementor 3.0.0 (/wp-content/themes/hello-elementor/style.css), All-in-One WP Migration 7.109 (/wp-content/plugins/all-in-one-wp-migration/readme.txt), Akismet 5.7.2 (/wp-content/plugins/akismet/readme.txt), Apache 2.4.68 (Server header).
- ticket: osTicket 1.18.4, nginx 1.22.1, Apache 2.4.68, PHP 8.2.33 (X-Powered-By).
- apex: nginx 1.22.1 on 443, nginx 1.25.5 on 8080 (two different versions on same host - inconsistent patching).

Business impact: streamlines attacker reconnaissance and makes drive-by exploit kits more likely to connect successfully.

Supporting evidence collected during the engagement:

- P4-blog/readme-body.txt
- P4-blog/version-markers.txt
- P6-verify/aio-readme.txt
- P6-verify/akismet-readme.txt
- P6-verify/wp-links-opml.txt
- P5-ticket/install-headers.txt

Recommendation

1. nginx: `server_tokens off;` in http {} block.
2. Apache: in /etc/apache2/conf-enabled/security.conf set `ServerTokens Prod` and `ServerSignature Off`.
3. PHP: in php.ini set `expose_php = Off`.
4. WordPress: remove the generator tag (`remove_action('wp_head','wp_generator');`), block /readme.html and /license.txt in the web server, and delete `wp-content/plugins/*/readme.txt` or deny access to `*/readme.txt`.
5. Ensure the apex host runs a single nginx version (currently 1.22.1 on :443 and 1.25.5 on :8080) and patch to the latest stable 1.26.x/1.27.x line.

M9: nginx 1.22.1 Running as Edge - Older Mainline Than Current Stable	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L
Target	• 51.79.166.148:443 • 51.79.166.209:443 • 51.79.167.208:80 • 51.79.167.208:443
References	• nginx security advisories • Nuclei nginx-eol template hit

Overview

nginx 1.22.1 is older than the current stable/mainline branches. While 1.22.x received backports for the HTTP/3 and ngx_http_mp4_module advisories, it is nearing community-support EOL. Running newer stable (1.26.x) or mainline (1.27.x) reduces the window of unpatched issues.

Details

nmap -sV and nuclei's nginx-eol template both tagged every nginx 1.22.1 instance. The 8080 backend on the apex IP runs 1.25.5, further indicating inconsistent nginx versions across the fleet.

Supporting evidence collected during the engagement:

- P6-crosscut/nuclei-focused.jsonl

Recommendation

Upgrade to nginx 1.26.x via the official nginx.org Debian repo, consolidate to a single version across all hosts, and set up unattended-upgrades for security patches.

M10: osTicket Setup Endpoint Also Discloses Internal Server Path	
Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://ticket.ausecurity.best/setup/inc/class.installer.php
References	CWE-209: Generation of Error Message Containing Sensitive Information

Overview

Fetching /setup/inc/class.installer.php directly causes PHP to emit a verbose fatal-error message that leaks the webroot path and the exact source line of a class file: `Fatal error: Uncaught Error: Undefined constant "INCLUDE_DIR" in /var/www/html/setup/inc/class.installer.php:16`.

Details

Because the file is included directly (bypassing the normal bootstrap that defines INCLUDE_DIR), PHP's `display_errors` is on and reveals internal filesystem layout (`/var/www/html/`) and the exposed script's absolute path. An attacker uses this to anchor further probes (e.g., LFI candidates, backup-naming conventions).

Business impact: aids an attacker's reconnaissance and increases the chance that any subsequent LFI/RFI vulnerability becomes exploitable. On its own, information disclosure only.

Supporting evidence collected during the engagement:

- P6-verify/installer-class.txt

Recommendation

1. In the osTicket php.ini (or PHP-FPM pool): `display_errors = Off`, `log_errors = On`, `error_log = /var/log/php/osticket-error.log`.
2. Remove the /setup/ directory altogether after installation (see critical installer finding).

M11: No Security Response Headers on Primary Website	
Score	4.7 (Medium)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:C/C:L/I:L/A:N
Target	• https://ausecurity.best/ • https://www.ausecurity.best/ • http://51.79.166.148:8080/
References	• OWASP Secure Headers Project • CWE-693: Protection Mechanism Failure

Overview

The primary marketing site does not set HSTS, Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy or any cross-origin isolation header. For a security-consulting firm this is particularly discordant: external auditors (and prospective clients using free scanners like Mozilla Observatory or securityheaders.com) will grade the site F.

Details

`curl -I https://ausecurity.best/` returns only `Server`, `Date`, `Content-Type`, `Content-Length`, `Last-Modified`, `ETag`, `Accept-Ranges`, `Connection`. No defensive headers. The blog does ship some (X-Frame-Options SAMEORIGIN, CSP frame-ancestors 'self', Referrer-Policy strict-origin-when-cross-origin) but still lacks HSTS and nosniff. The osTicket responses include zero security headers.

Business impact: enables clickjacking of the home page in iframes, allows browsers to MIME-sniff content types, and does not force transport security on returning visitors.

Supporting evidence collected during the engagement:

- P2-apex/headers-root.txt
- P4-blog/headers-root.txt
- P5-ticket/install-headers.txt

Recommendation

Add to the nginx server block for all three sites (once a valid TLS cert is in place for the apex):

```
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload"
always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Permissions-Policy "geolocation=(), camera=(), microphone=()" always;
add_header Content-Security-Policy "default-src 'self'; img-src 'self' https: data:;
```

```
style-src 'self' https://fonts.googleapis.com 'unsafe-inline'; font-src https://
fonts.gstatic.com; script-src 'self'; frame-ancestors 'self'" always;
```

Then validate at https://securityheaders.com/.

L1: Missing CAA DNS Record	
Score	3.7 (Low)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:N
Target	ausecurity.best
References	RFC 8659 (CAA), CA/B Forum Baseline Requirements §3.2.2.8

Overview

The ausecurity.best zone publishes no CAA records. Any public Certificate Authority is permitted to issue a certificate for the domain. In the event of a registrant account compromise or an incorrectly validated DCV at any CA, a mis-issued cert would be accepted by every browser.

Details

`dig CAA ausecurity.best` returns no answer. DNSSEC is correctly enabled on the zone, so a CAA record would be cryptographically protected once added.

Supporting evidence collected during the engagement:

- P3-dns/caa.txt

Recommendation

Publish CAA at the apex pinning Let's Encrypt (the issuer actually used for blog/ticket):

```
ausecurity.best. 3600 IN CAA 0 issue "letsencrypt.org"
ausecurity.best. 3600 IN CAA 0 iodef "mailto:security@ausecurity.best"
```

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerabilty assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section